

PredictaPK: Real-Time Market Valuator

Interview Preparation & Technical Documentation

1. Project Overview

PredictaPK is a full-stack, AI-powered web application designed to predict and analyze market prices for vehicles (cars, bikes) and real estate (buying, renting) in Pakistan. The platform not only provides highly accurate price predictions but also leverages Explainable AI (XAI) to tell the user *why* a specific price was predicted, alongside historical trends and budget-based recommendations.

2. System Architecture

The application follows a decoupled, modern architecture:

- **Frontend:** Built with **Next.js 14+ (React)**, using **Tailwind CSS** and **Framer Motion** for a highly responsive, animated, and interactive user interface. Deployed on **Vercel**.
- **Backend:** Built with **Python & FastAPI**, serving the machine learning models via RESTful API endpoints. Deployed on **Hugging Face Spaces** for robust, free-tier compute optimized for Python data science workloads.
- **Machine Learning Layer:** Built with **scikit-learn**, utilizing ensemble learning techniques.

3. Machine Learning Backend (Deep Dive)

3.1 Data Preprocessing Pipeline

The training script (`train_models.py`) handles raw CSV datasets for cars, bikes, and properties. It utilizes a `scikit-learn Pipeline` and `ColumnTransformer` to ensure data leakage is prevented and transformations are reproducible.

- **Numerical Features** (e.g., year, mileage, engine capacity, area): Processed using `SimpleImputer(strategy='median')` to handle any missing values robustly against outliers.
- **Categorical Features** (e.g., make, model, city, transmission): Processed securely using a pipeline of `SimpleImputer(strategy='most_frequent')` followed by `OneHotEncoder(handle_unknown='ignore')`.

3.2 The Prediction Model: Random Forest Regressor

The core prediction engine uses the **RandomForestRegressor**.

- **Why Random Forest?** Vehicle and real estate pricing is highly non-linear with complex feature interactions (e.g., the combination of a specific make, model, and year impacts price differently than each feature individually). Random Forests naturally capture these multi-dimensional interactions without requiring heavy manual feature engineering.
- **Hyperparameters Used:**
 - `n_estimators=100`: Trains 100 individual decision trees and averages their outputs to prevent overfitting.
 - `max_depth=25`: Allows trees to grow deep enough to capture complex patterns but caps them to prevent memorizing the noise in the dataset.
 - `min_samples_split=10`: Ensures a node must have at least 10 samples before splitting, further acting as regularization against overfitting.

3.3 Explainable AI (XAI) with SHAP

One of the standout features of this project is the integration of **SHAP (SHapley Additive exPlanations)** in `main.py`.

- **How it works:** When a prediction is made, the backend uses `shap.TreeExplainer` on the Random Forest model. It calculates the exact marginal contribution of every single input feature to the final predicted price.
- **Business Value:** Instead of a "black box" prediction, the user sees exactly how much value factors like "Transmission: Automatic" or "Assembly: Imported" add or subtract from the base valuation.

3.4 Budget Recommendation Algorithm

The `/api/recommend` endpoint is a custom data-filtering algorithm. It doesn't use the ML model directly; instead, it searches the underlying dataset.

- It applies user filters (e.g., City, Make, minimum area).
 - It initiates a search with a +15% margin above the user's budget.
 - If no results are found, it gracefully falls back by widening the search radius (to +30%, then +50%) to ensure the user always receives the closest available options, sorting them by absolute difference from the target budget.
-

4. Frontend & API Integration

4.1 Key Endpoints

- `GET /api/options/{service}`: Dynamically returns the available features (makes, models, cities) directly from the trained datasets so the frontend dropdowns are perfectly in sync with what the model understands.
- `POST /api/predict`: Receives user input, transforms it, predicts the price, calculates SHAP values, generates a synthetic 5-year price trend, and returns the aggregated JSON response.
- `POST /api/recommend`: Returns top 5 dataset matches based on budget.

4.2 Frontend Highlights

- **State Management**: Uses React `useState` and `useEffect` hooks extensively to manage form data, history, and comparison states. Re-fetches engine capacities dynamically based on selected bike models.
 - **Styling**: Uses a customized CSS gradient background with dynamic noise and particle animations (`framer-motion`), providing a premium "glassmorphism" aesthetic.
 - **Comparison & PDF Reports**: Implements `html-to-image` and `jsPDF` completely client-side. It captures the DOM nodes of the result cards and converts them into an organized, downloadable PDF report.
-

5. Potential Interview Questions to Prepare For

Q: Why did you choose Random Forest over Linear Regression?

A: Linear regression assumes a straight-line relationship between features and price, which isn't true for real estate or cars (prices depreciate on a curve, and location impacts price non-linearly). Random forest captures these non-linear patterns effectively.

Q: How do you handle a scenario where a user asks for a car model the model hasn't seen?

A: The frontend dynamically fetches `make_model_tree` from `/api/options`, ensuring users can only select data the model was trained on. Additionally, the `OneHotEncoder` uses `handle_unknown='ignore'`, so if an unseen category somehow slips through the API, the model ignores it rather than crashing.

Q: Tell me about the CORS and Deployment setup.

A: The frontend is static and hosted on Vercel. The backend is heavy (running ML models) and hosted on Hugging Face Spaces. I used `CORSMiddleware` in `FastAPI` to allow cross-origin requests from the Vercel app to the Hugging Face domain, connecting them securely.